ACCOUNT MANAGEMENT / TEST RECORD

Proantic

CSRF testing report

ASSESSMENT OUTCOME

No CSRF vulnerability found in the tested cases.

I tested the checks used when changing an account's email and password. The attempted email-token bypasses were rejected. The password feature also rejected an incorrect current password.

TESTER
Haroon Shahid

CAPTURED TEST DATE
7 November 2026 (GMT / UTC)

TOOLS
Burp Suite and Chrome DevTools

RECORD
CSRF-2026-01 / public edition 3.0

THE EXACT TARGET

Website: https://www.proantic.com
Email page: /en/monespace.php?mode=email
Password page: /en/monespace.php?mode=pwd

Shared update endpoint

`/script/member/script-modifier-member-ajax-csrf.php`

My test accounts: main account shown as Account A, later Account A (updated email); second account shown as Account B.

READING GUIDE

How I tested / simple terms	02
All test results	03
16 full screenshots with explanations	04-19
Final conclusion and limits	20
Full request text from the testing notes	21-22

01 / HOW THE TESTING WORKED

The process, in simple words.

FIVE WORDS USED IN THIS REPORT

Request: A message that the browser sends to the website, such as "change my email".

Response: The website's answer to that message.

Session: The login that the website recognises from the browser's cookies.

CSRF token: A hidden value sent with the form. The website checks whether it is valid for the request.

Working comparison: A normal request that succeeds. I compare edited requests against it.

WHAT I DID

01 **Open the feature.** I opened the email-change page while logged in to my test account.

02 **Check a normal update.** A normal browser request returned success. An early replay of a request from HTTP history returned a token error.

03 **Capture before sending.** I intercepted a fresh request, sent it to Burp Repeater, then dropped the browser request. The fresh request worked in Repeater.

04 **Change one input at a time.** I kept the login cookies and edited the token. I tried an empty value, no token, an arbitrary value and a one-character change.

05 **Compare two accounts.** I put the second account's token into the main account's request. I also checked repeated use before reload and old/new tokens after reload.

06 **Check the other cases.** I tested the password form with correct and incorrect current passwords. I also sent an email-change request using GET instead of POST.

HOW I READ THE RESPONSE

HTTP 200 means the server answered. It does not mean the account change worked. I read the success or error message inside the response body.

02 / COMPLETE TEST RECORD

What happened in each test.

E-numbers refer to the original screenshot numbers. "Written notes" means that the result was described in the supplied test log.

WHAT I TESTED	WHAT THE SITE RETURNED	EVIDENCE
Normal email request	Success response	E04
Empty email token	Security error	Written notes
Email token removed	Security error	E05
Token set to invalid123	CSRF token error	E06
One token character changed	CSRF token error	Written notes
Main login + other account's token	CSRF token error	E07-E09 + notes
Same token sent twice, no reload	Both returned success	E10, E11
Old token sent after reload	CSRF token error	E12
New token sent after reload	Success response	E13, E14
Correct current password	Password success response	E15, E16
Incorrect current password	Current-password error	E17
Password token reuse / reload	Same pattern reported	Written notes
Email request changed to GET	Security error; see limit	E18

THE MAIN LESSON

The token worked more than once before reload. After reload, the old token failed and the new token worked. It was reusable, not single-use.

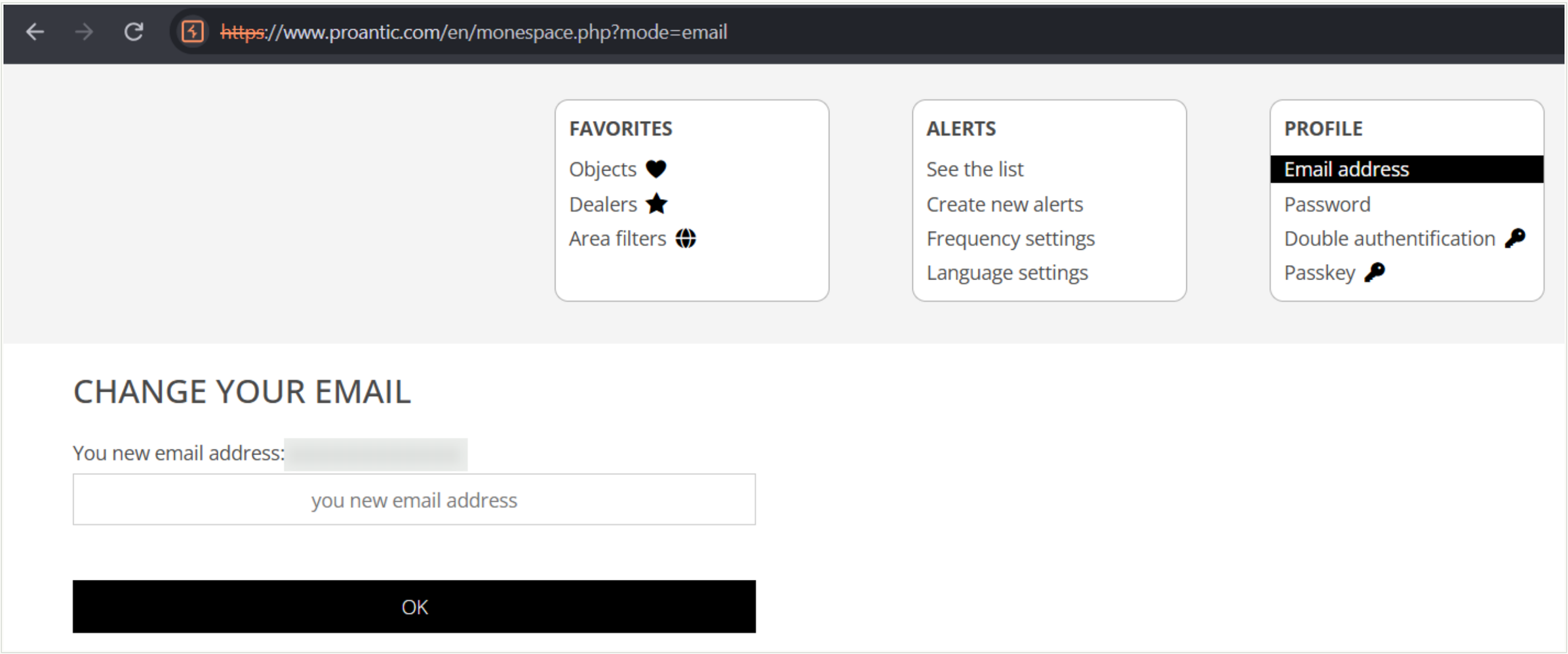
GET test limit: E18 uses a token also visible in the password-form evidence. No working email POST with that exact token is shown. The evidence proves that this GET request failed; it does not prove why.

E03 / EMAIL CHANGE

FEATURE IDENTIFIED

I opened the email-change form.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

I opened the account page at /en/monespace.php?mode=email. The main test account showed Account A.

WHAT THE WEBSITE SAID

The page offered a field for a new email address and an OK button.

WHAT THIS MEANS

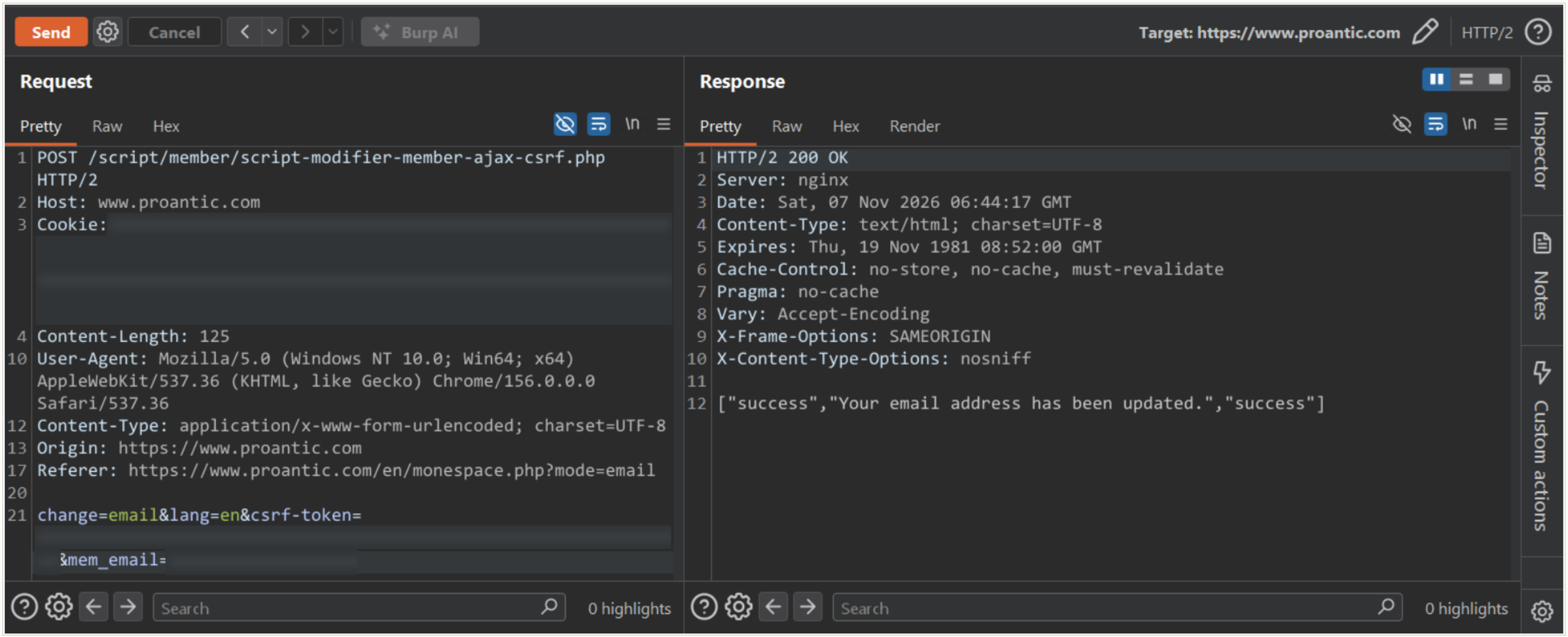
This identifies the first feature I tested. The account email later changed during testing.

E04 / EMAIL CHANGE

WORKING COMPARISON

A normal request worked.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

I sent a POST request with the main account's cookies and its valid csrf-token. The requested email was Account A (updated email).

WHAT THE WEBSITE SAID

The response body said: "Your email address has been updated."

WHAT THIS MEANS

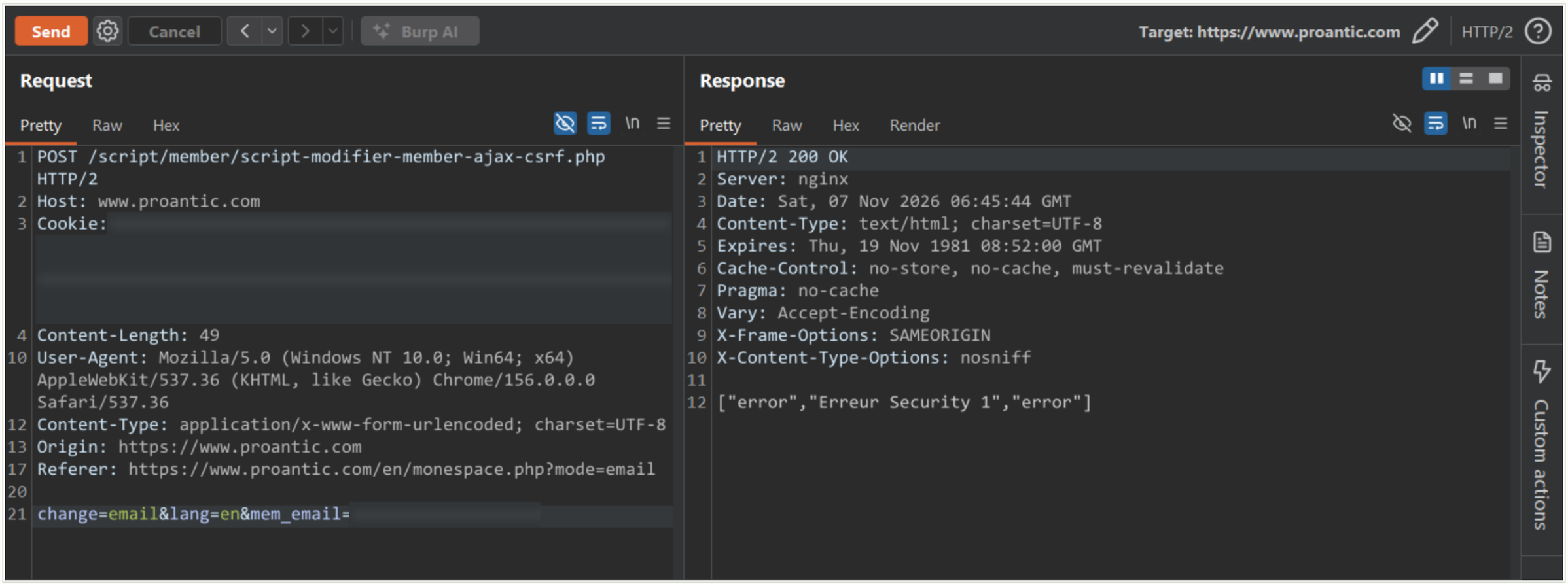
This gave me a working request to compare with the edited requests. A later browser screenshot also shows Account A (updated email).

E05 / EMAIL CHANGE

REQUEST REJECTED

Removing the token failed.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

I removed the entire csrf-token parameter and sent the email-change request with the same logged-in context.

WHAT THE WEBSITE SAID

The response body said: "Erreur Security 1". The application rejected the request.

WHAT THIS MEANS

Leaving out the token did not bypass the check. In a separate check in my written notes, an empty token returned the same error.

E06 / EMAIL CHANGE

REQUEST REJECTED

A made-up token failed.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT

Send Cancel Burp AI Target: https://www.proantic.com HTTP/2

Request

Pretty Raw Hex

```
1 POST /script/member/script-modifier-member-ajax-csrf.php
  HTTP/2
2 Host: www.proantic.com
3 Cookie:
4 Content-Length: 71
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/156.0.0.0
  Safari/537.36
12 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
13 Origin: https://www.proantic.com
17 Referer: https://www.proantic.com/en/monespace.php?mode=email
20
21 change=email&lang=en&csrf-token=invalid123&mem_email=
```

Search 0 highlights

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Server: nginx
3 Date: Sat, 07 Nov 2026 06:47:14 GMT
4 Content-Type: text/html; charset=UTF-8
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate
7 Pragma: no-cache
8 Vary: Accept-Encoding
9 X-Frame-Options: SAMEORIGIN
10 X-Content-Type-Options: nosniff
11
12 ["error","Error CSRF Token","error"]
```

Search 0 highlights

Inspector Notes Custom actions

WHAT I DID

I replaced the real token with invalid123 and sent the request.

WHAT THE WEBSITE SAID

The response body said: "Error CSRF Token".

WHAT THIS MEANS

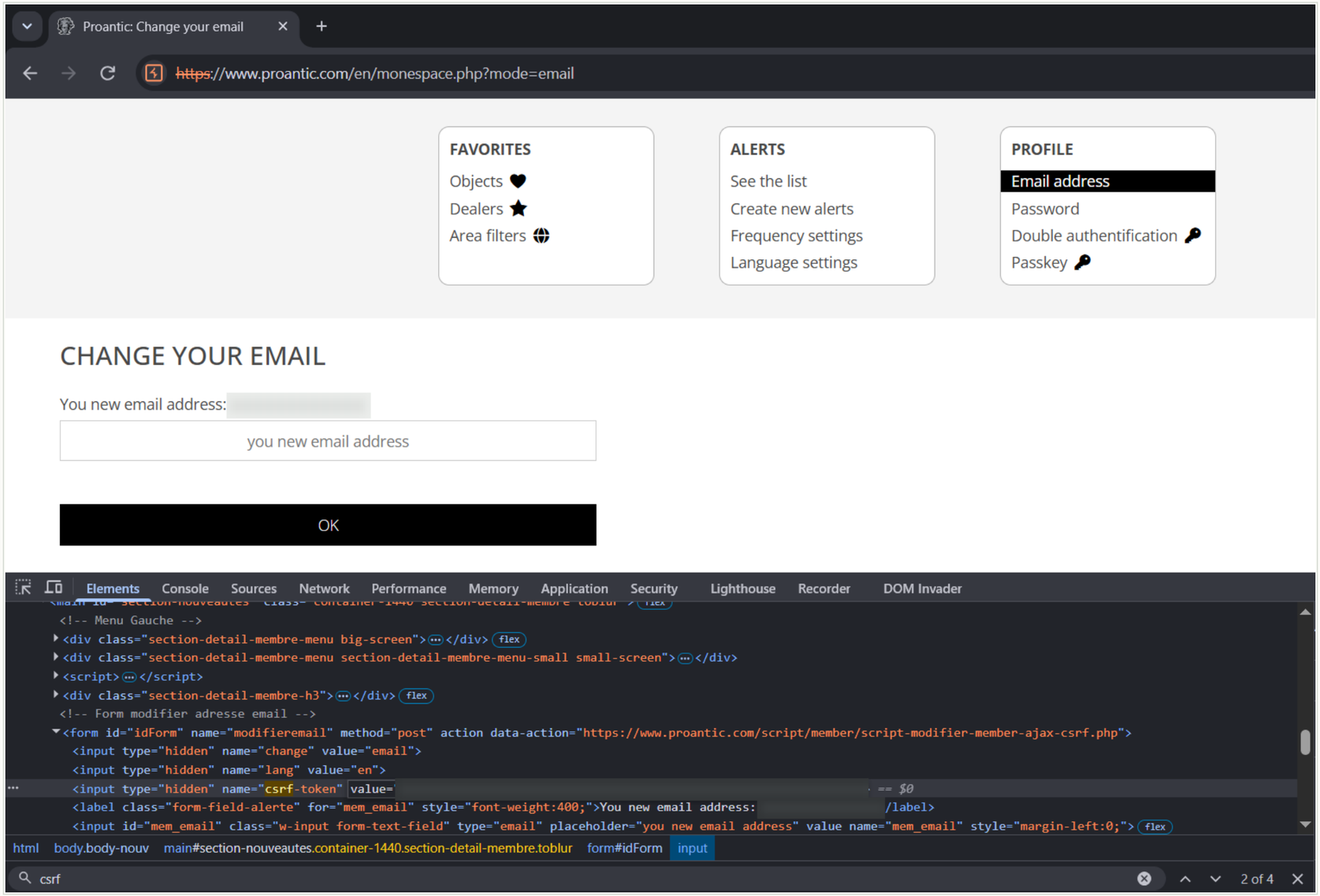
A token field containing any random text was not enough. My written notes also record rejection after changing one character of a real token.

E07 / TWO-ACCOUNT CHECK

TOKEN SOURCE

I found the main account's token.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

I inspected the email form in Chrome DevTools while using the main account, Account A.

WHAT THE WEBSITE SAID

The page contained a hidden input named csrf-token. Its value is blurred in this public screenshot.

WHAT THIS MEANS

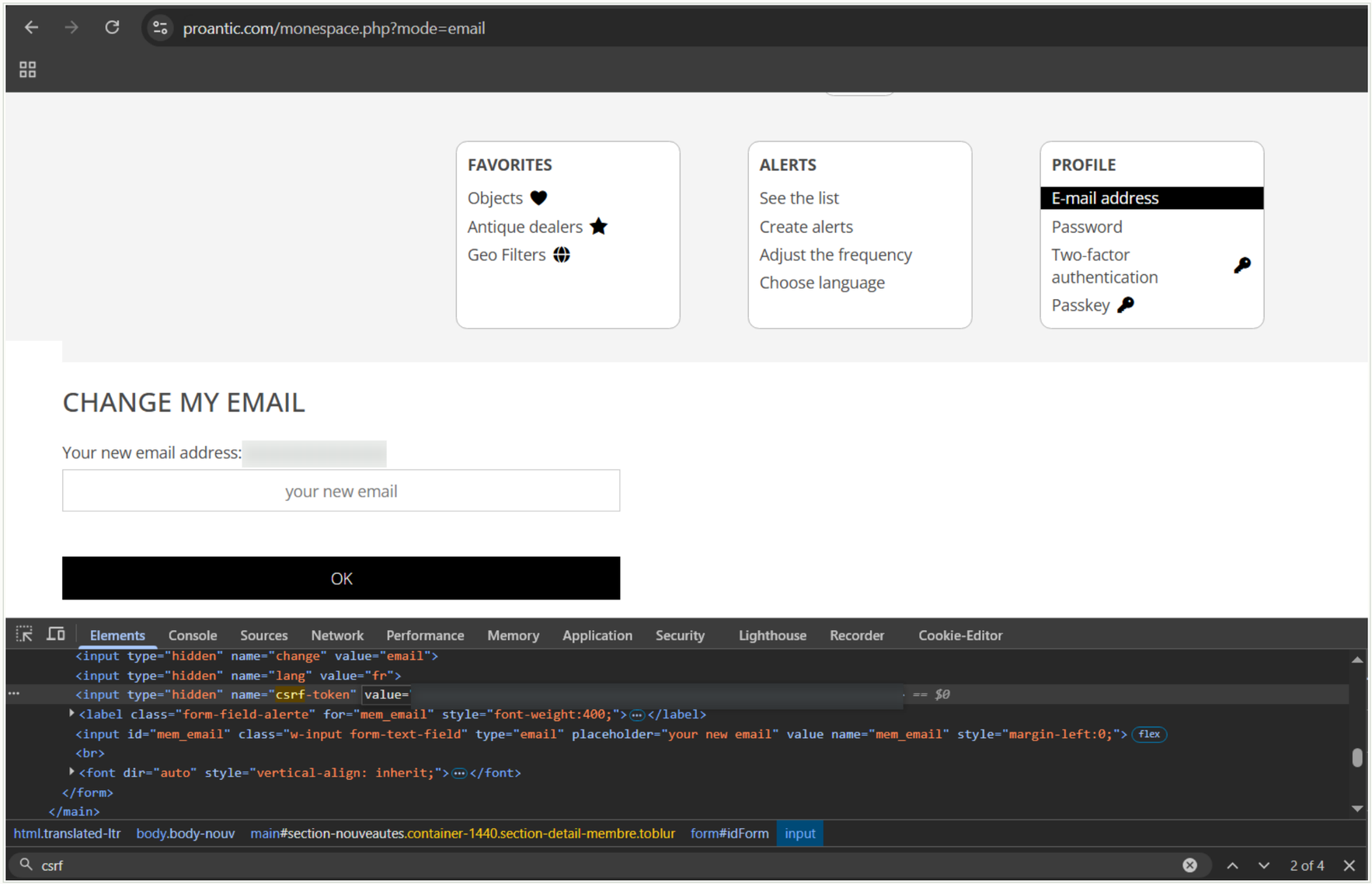
This links the main account's email form to the token used in its working requests. This page alone does not show a new successful submission.

E08 / TWO-ACCOUNT CHECK

SECOND TOKEN SOURCE

I found the second account's token.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

I opened the email form for my second test account, Account B, and inspected its hidden token.

WHAT THE WEBSITE SAID

This account had a different csrf-token value.

WHAT THIS MEANS

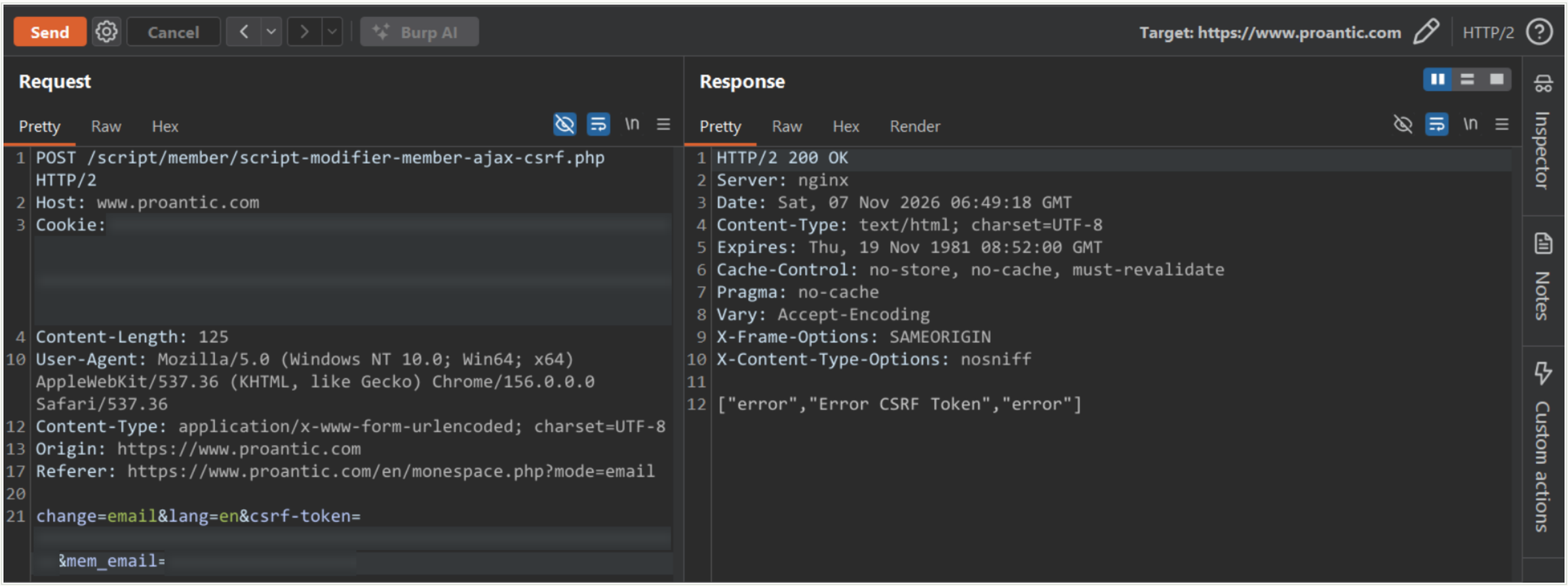
I now had tokens from two different account contexts. I confirmed in my testing notes that both were still valid during the substitution test.

E09 / TWO-ACCOUNT CHECK

CROSS-ACCOUNT REJECTED

The other account's token failed.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

I kept the main account's login cookies and replaced only its token with the second account's token. Neither form was reloaded.

WHAT THE WEBSITE SAID

The response body said: "Error CSRF Token".

WHAT THIS MEANS

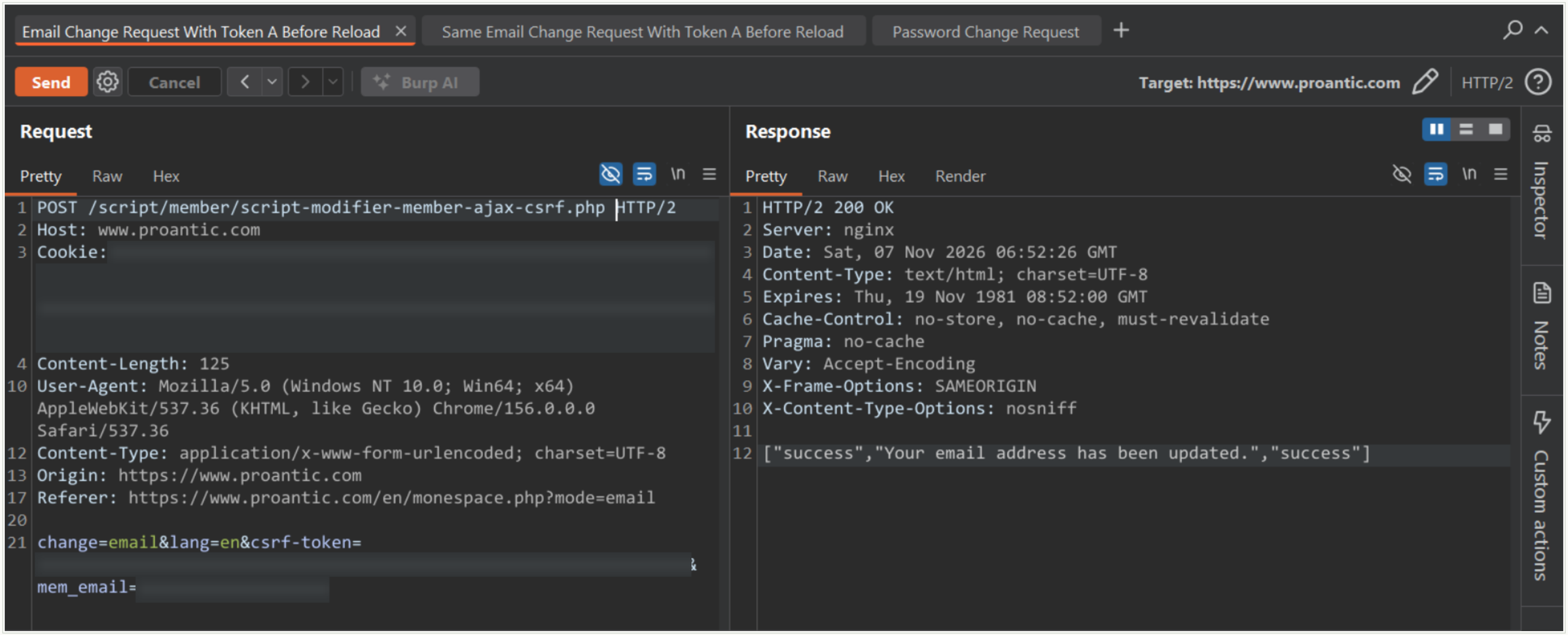
The other account's token did not authorise this request. Both-token validity is confirmed in my notes; a separate successful second-account request was not captured.

E10 / TOKEN BEFORE RELOAD

FIRST SUCCESS RESPONSE

The token worked before reload.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

I sent an email-change request with the main account's valid token and did not reload the account page.

WHAT THE WEBSITE SAID

The response body reported a successful email update at 06:52:26 GMT.

WHAT THIS MEANS

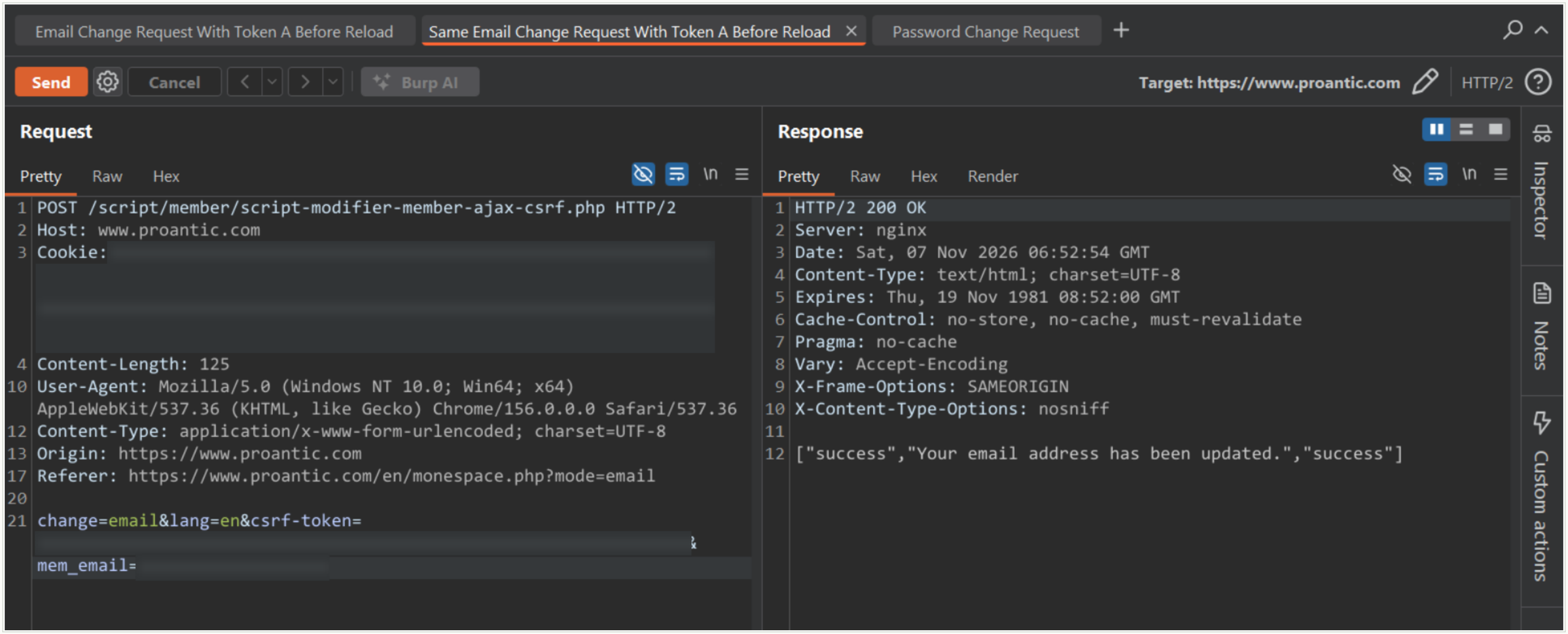
This is the first request in the token-reuse comparison. The next screenshot shows the same token being accepted again.

E11 / TOKEN BEFORE RELOAD

TOKEN REUSE OBSERVED

The same token worked again.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

I sent another request with the same token and login cookies, without reloading the page.

WHAT THE WEBSITE SAID

The response again reported a successful email update, at 06:52:54 GMT.

WHAT THIS MEANS

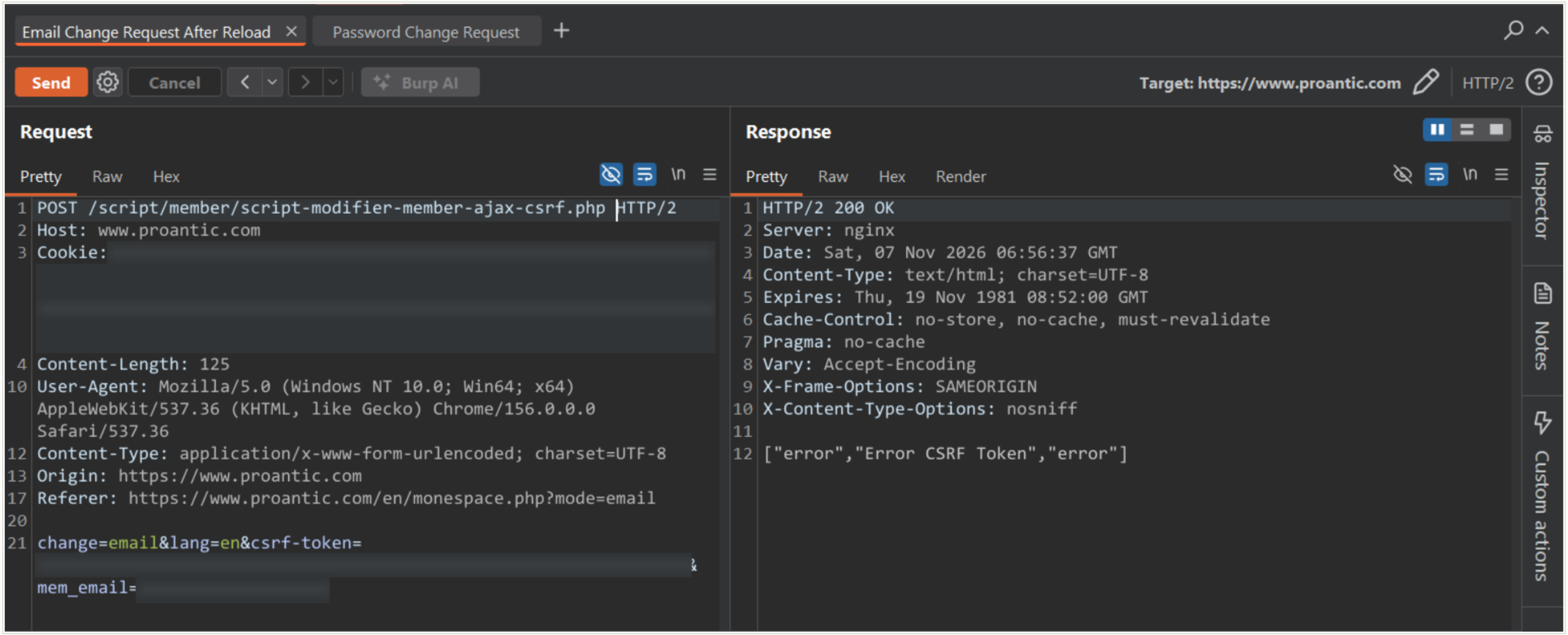
The token was reusable. This corrected the earlier idea that every successful request consumed the token. Two success responses are shown; this second screenshot alone does not prove another stored email change.

E12 / TOKEN AFTER RELOAD

OLD TOKEN REJECTED

The old token failed after reload.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

I reloaded the account page, then sent the earlier request again while keeping the main account's session.

WHAT THE WEBSITE SAID

The old token returned "Error CSRF Token" at 06:56:37 GMT.

WHAT THIS MEANS

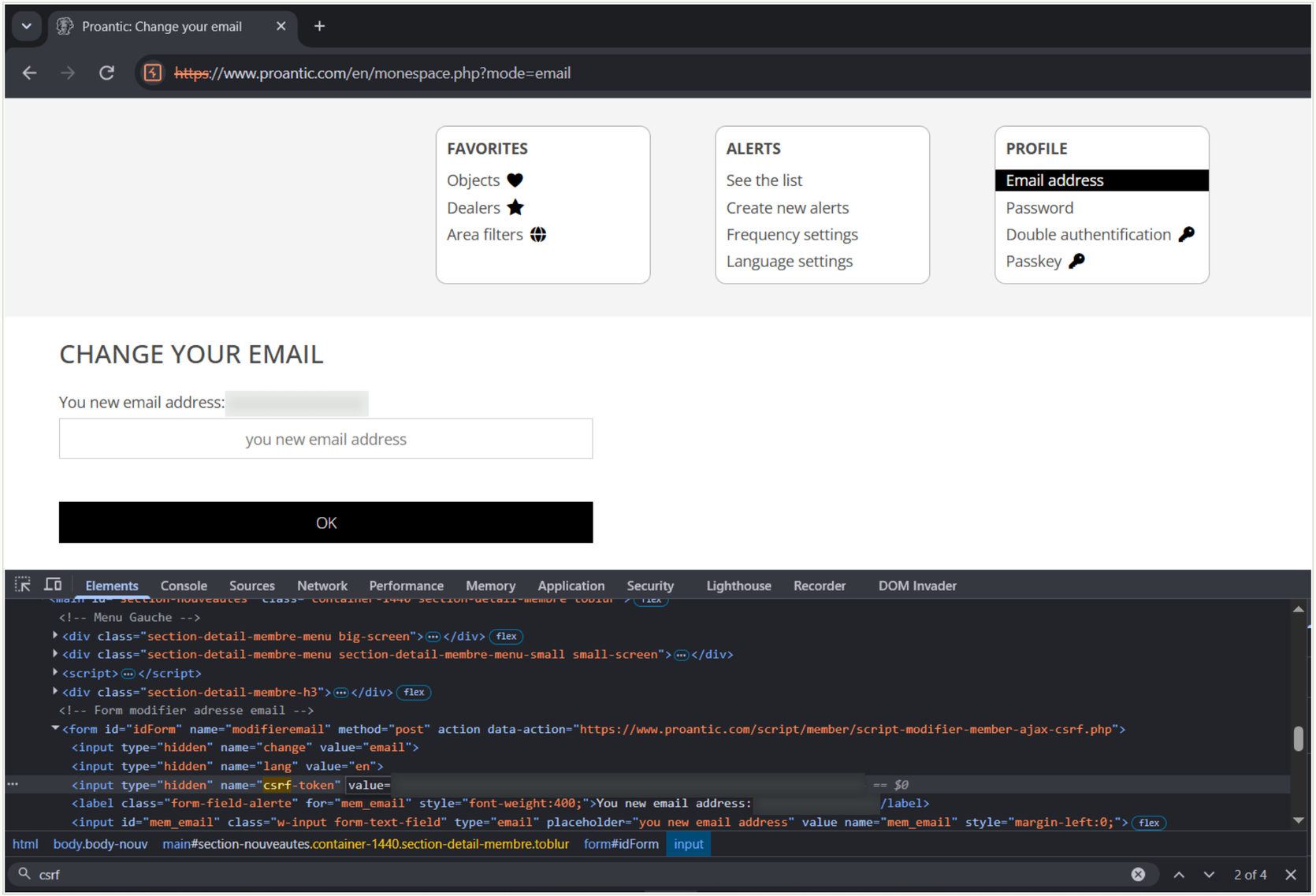
The old value was no longer accepted after reload. This is different from the token becoming invalid after one use.

E13 / TOKEN AFTER RELOAD

NEW TOKEN CAPTURED

The page showed a new token.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

After reload, I inspected the email form again in Chrome DevTools.

WHAT THE WEBSITE SAID

The page showed Account A (updated email) and a token different from the earlier one.

WHAT THIS MEANS

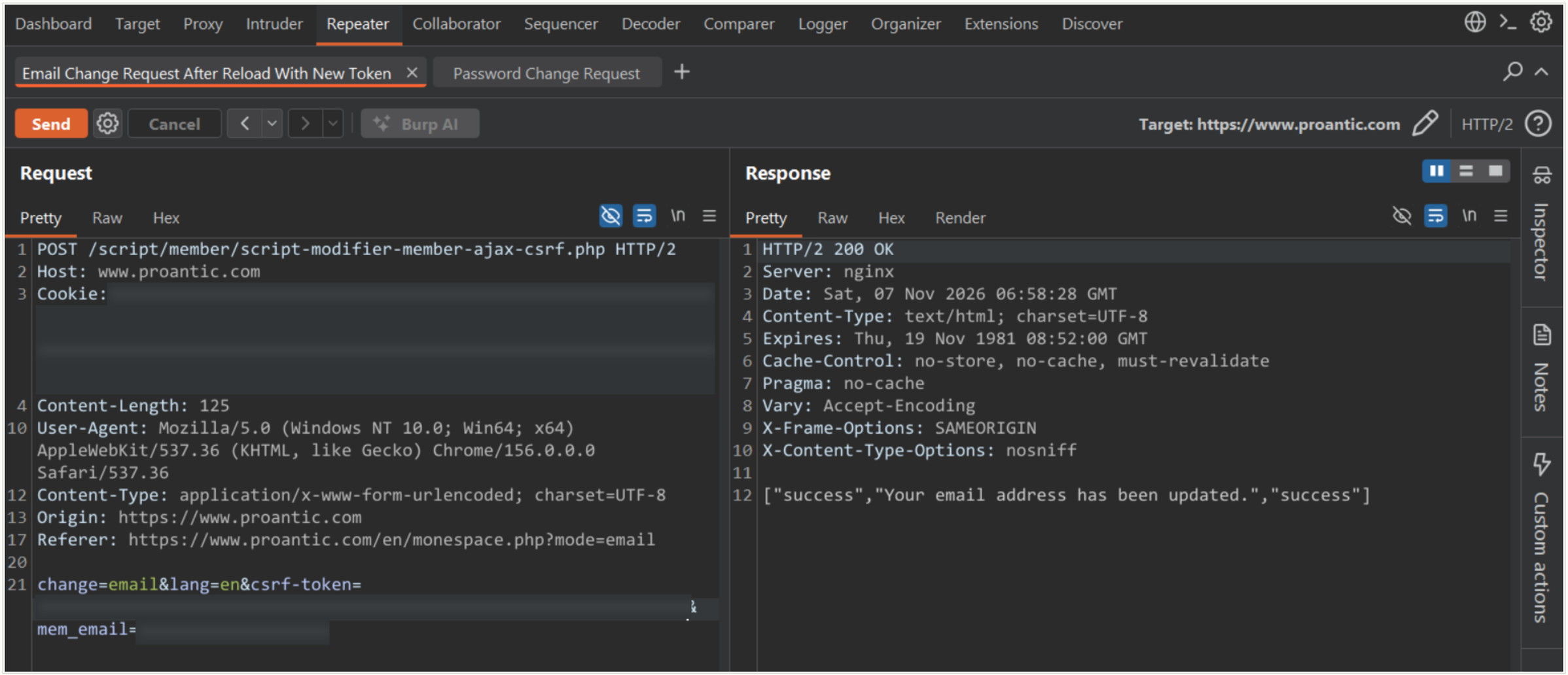
The browser view supports the email update and shows the replacement token. The next request checks whether this new value works.

E14 / TOKEN AFTER RELOAD

NEW TOKEN ACCEPTED

The new token worked.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

I kept the same main-account session and used the replacement token from the reloaded page.

WHAT THE WEBSITE SAID

The response reported a successful email update at 06:58:28 GMT.

WHAT THIS MEANS

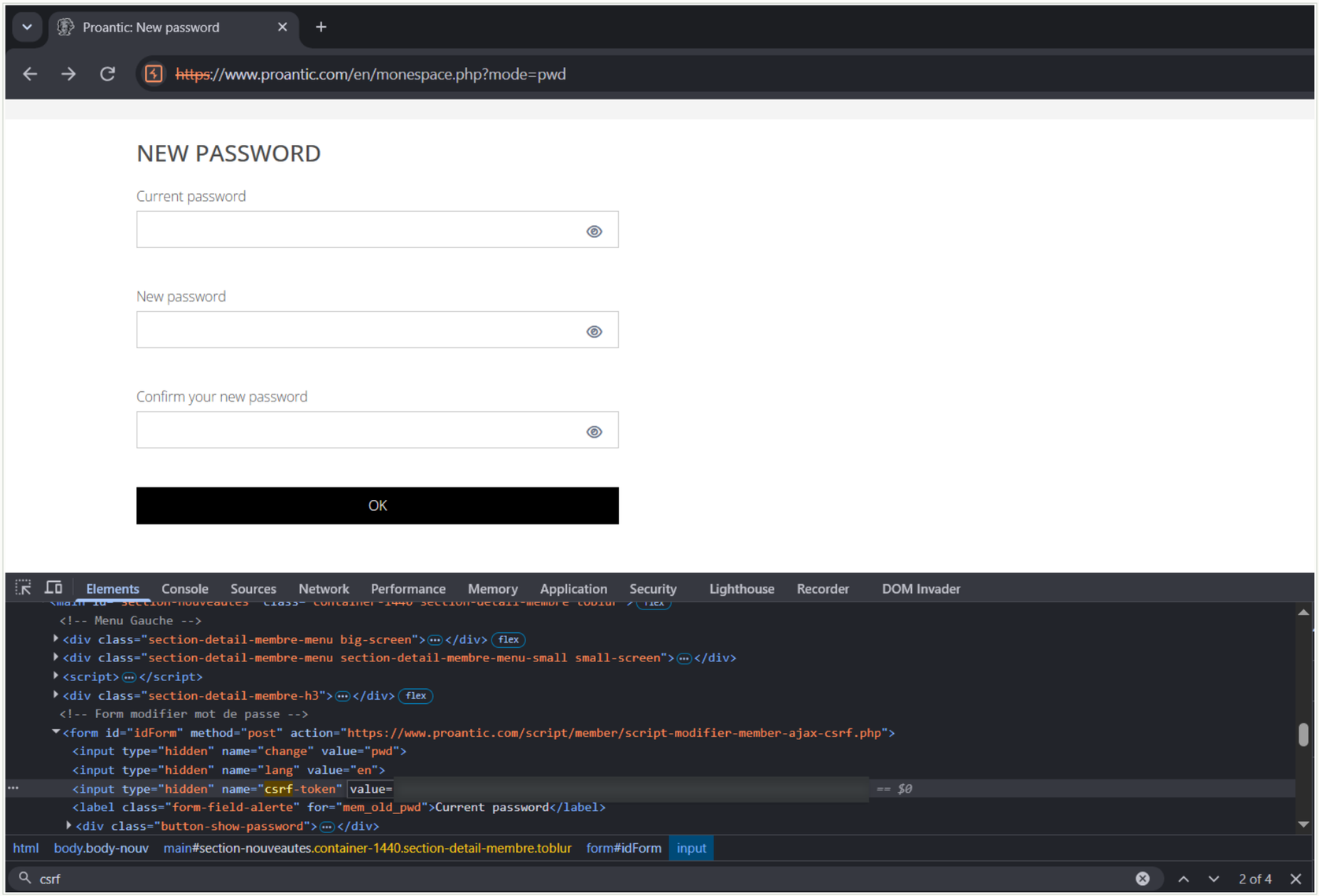
The old token failed, but the replacement worked. Together, these tests support token replacement when the page was reloaded. They do not reveal the server's internal code.

E15 / PASSWORD CHANGE

SECOND FEATURE IDENTIFIED

I opened the password form.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

I opened /en/monespace.php?mode=pwd and inspected the form.

WHAT THE WEBSITE SAID

The form asked for Current password, New password and Confirm your new password. It also included a hidden csrf-token.

WHAT THIS MEANS

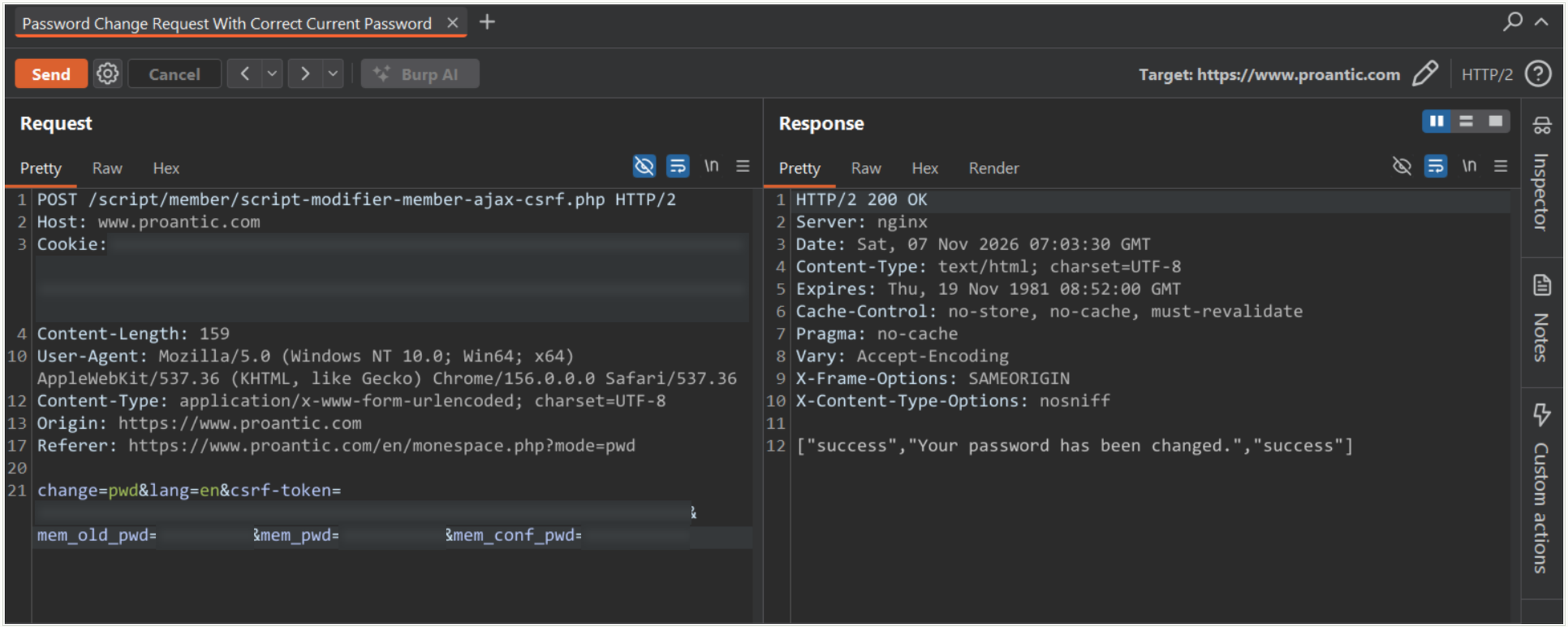
The password action had an extra input to check: the current password. It used the same update endpoint with change=pwd.

E16 / PASSWORD CHANGE

PASSWORD SUCCESS RESPONSE

The correct current password worked.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

I sent the password-change request using the correct current password, a token and matching new-password fields.

WHAT THE WEBSITE SAID

The response body said: "Your password has been changed."

WHAT THIS MEANS

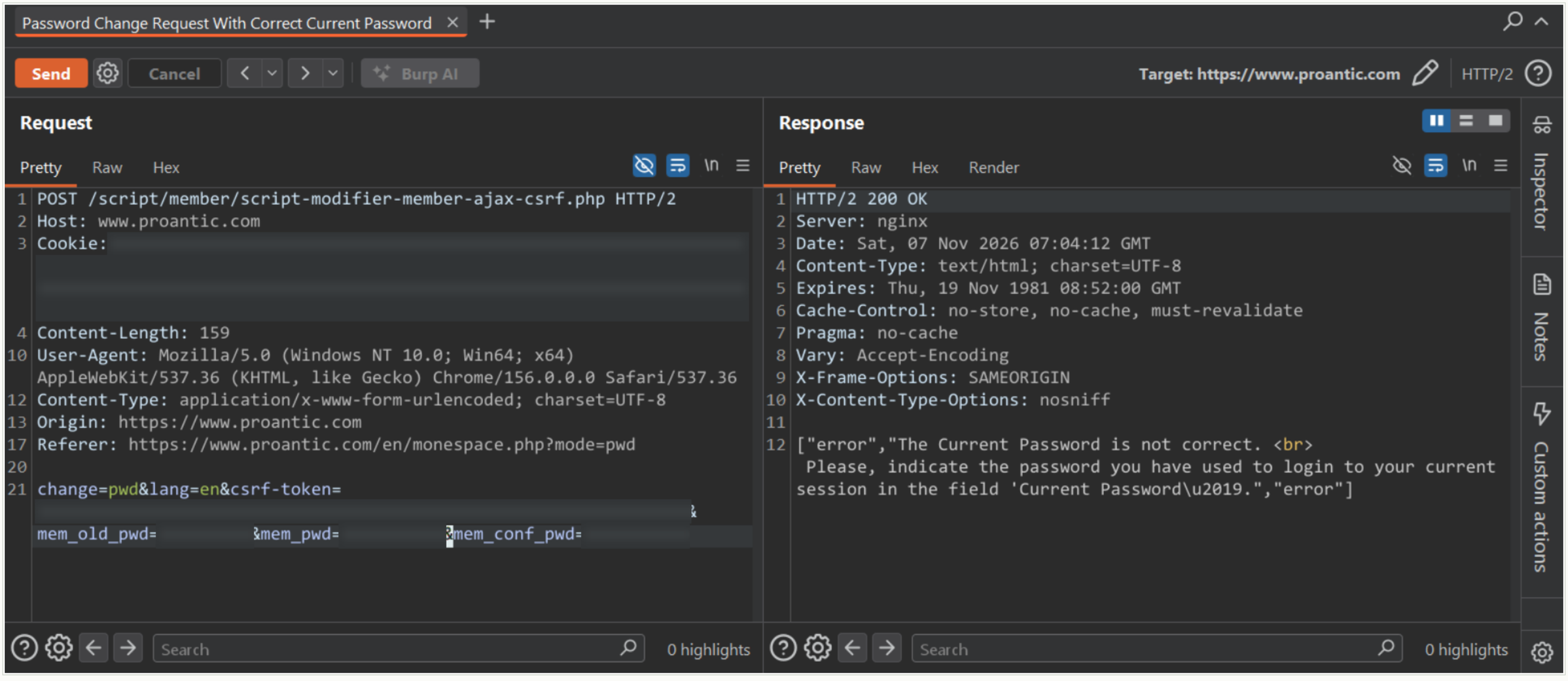
The application reported that the normal password update worked. A separate logout-and-login check with the new password was not included in the evidence.

E17 / PASSWORD CHANGE

CURRENT PASSWORD REJECTED

An incorrect current password failed.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

I checked a request whose current-password value was incorrect for the account at that point in the test.

WHAT THE WEBSITE SAID

The response said: "The Current Password is not correct." The old Repeater tab title still says Correct Current Password; the result is the error on the right.

WHAT THIS MEANS

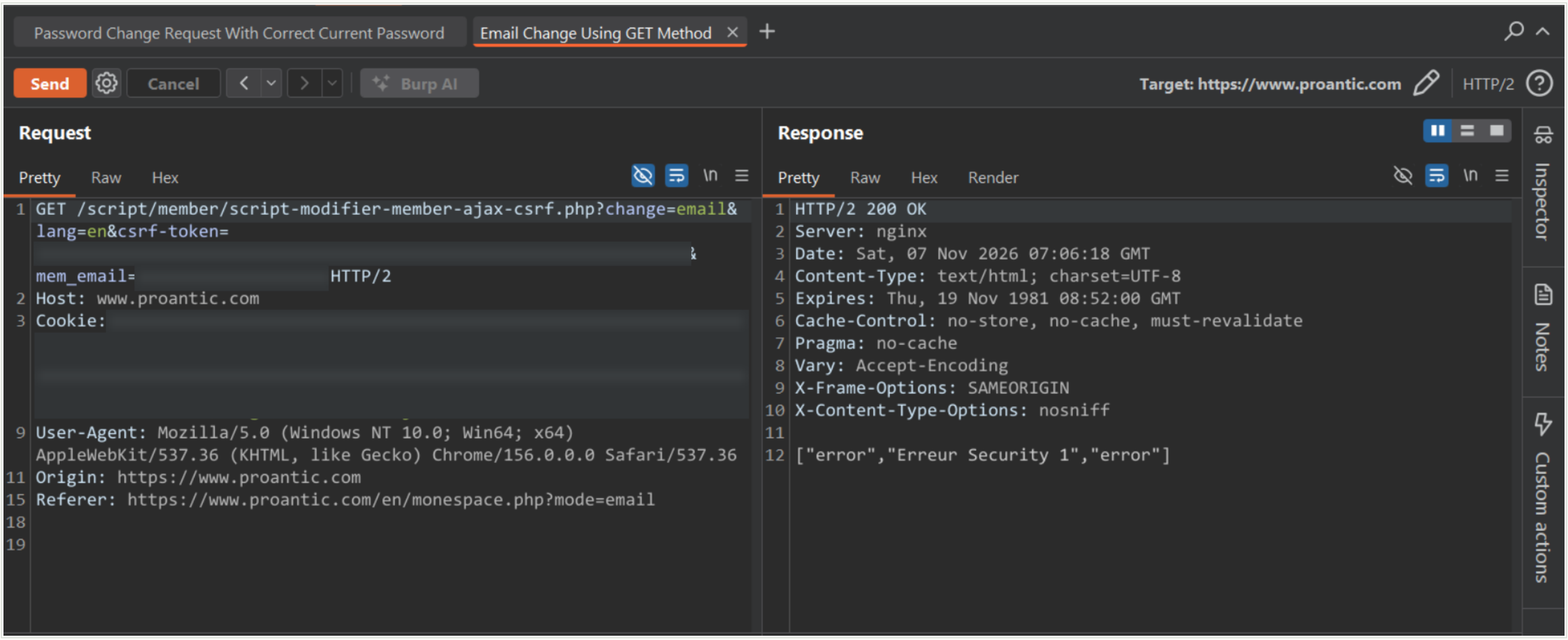
The current-password check rejected this request. After a successful password change, the old password is no longer current. This error alone says nothing about whether the token was still valid.

E18 / REQUEST METHOD CHECK

GET REQUEST REJECTED

The tested GET request failed.

FULL UPLOADED FRAME / SENSITIVE VALUES BLURRED / ZOOM FOR SMALL TEXT



WHAT I DID

I changed the email request from POST to GET and moved the form fields into the URL query string.

WHAT THE WEBSITE SAID

The response body said: "Erreur Security 1".

WHAT THIS MEANS

This GET request was rejected. Its token also appears in the password-form evidence. No successful email POST with that exact token is shown, so this does not isolate a method-only check.

03 / FINAL ASSESSMENT

What I can conclude.

RESULT

No CSRF vulnerability found in the tested cases.

The tested email requests rejected empty, missing, incorrect and other-account tokens. The valid email token worked repeatedly until a page reload replaced it in the observed sequence. The password feature returned success with the correct current password and an error with an incorrect one.

A token working more than once is not automatically a bug. OWASP describes token designs that allow reuse during a login session [1]. The question is whether an attacker can make an unwanted change without the required valid information.

WHAT THESE TESTS DO NOT PROVE

Cross-site browser attack. No successful attack page on a different site was demonstrated.

Other browser protections. Origin, Referer, SameSite cookies and X-Requested-With enforcement were not separately established.

Exact token binding. The other account's token failed. Two separate logins of the same account were not tested, so account binding and session binding cannot be separated.

Password token coverage. Token reuse and reload behaviour were reported for the password feature. Separate missing, empty, invalid and other-account token screenshots were not supplied for that feature.

Saved account state. Most captures show response messages. E13 also shows the updated email in the browser. A new-login check after the password update was not supplied.

Server implementation. The tests do not reveal token storage, generation quality, validation order or every event that can replace a token.

Classification: assessment record; no confirmed vulnerability, exploit impact or CVSS score. This conclusion applies to the cases documented here, not every feature on Proantic.

EVIDENCE AND REFERENCE

Compiled from 16 original screenshots and the supplied testing notes. The evidence records testing on 7 November 2026; no live retesting was performed for this revision.

[1] OWASP, Cross-Site Request Forgery Prevention Cheat Sheet. Read the synchronizer-token guidance.

APPENDIX / COMPLETE REQUEST TEXT

Full email request

Earlier capture from the testing notes. Every header and field is retained; sensitive values use labels. Original Content-Length values are kept, so placeholder lengths differ.

```
POST /script/member/script-modifier-member-ajax-csrf.php HTTP/2
Host: www.proantic.com
Cookie: cookie_analytic=[COOKIE_VALUE]; _ga=[COOKIE_VALUE]; mem_form_email=[COOKIE_VALUE]; PHPS
ESSID=[COOKIE_VALUE]; mem_id=[COOKIE_VALUE]; memsid=[COOKIE_VALUE]; _ga_YP51KSLLWR=[COOKIE_VALU
E]
Content-Length: 125
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Not:A-Brand";v="8", "Chromium";v="156"
Sec-Ch-Ua-Mobile: ?0
X-Requested-With: XMLHttpRequest
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Ch
rome/156.0.0.0 Safari/537.36
Accept: */*
Content-Type: application/x-www-form-urlencoded; charset=UTF-8
Origin: https://www.proantic.com
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://www.proantic.com/en/monespace.php?mode=email
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

change=email&lang=en&csrf-token=[CSRF_TOKEN]&mem_email=[TEST_EMAIL_A]
```

REPORTED RESPONSE BODY

```
["success","Your email address has been updated.","success"]
```

This earlier fresh request worked in Repeater after the intercepted browser request was dropped. Later repeat-use and reload tests corrected the initial single-use-token idea.

APPENDIX / COMPLETE REQUEST TEXT

Full password request

Earlier capture from the testing notes. Every header and field is retained; sensitive values use labels. Original Content-Length values are kept, so placeholder lengths differ.

```
POST /script/member/script-modifier-member-ajax-csrf.php HTTP/2
Host: www.proantic.com
Cookie: cookie_analytic=[COOKIE_VALUE]; _ga=[COOKIE_VALUE]; PHPSESSID=[COOKIE_VALUE]; mem_id=[C
OOKIE_VALUE]; memsid=[COOKIE_VALUE]; _ga_YP51KSLLWR=[COOKIE_VALUE]
Content-Length: 158
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Not:A-Brand";v="8", "Chromium";v="156"
Sec-Ch-Ua-Mobile: ?0
X-Requested-With: XMLHttpRequest
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Ch
rome/156.0.0.0 Safari/537.36
Accept: */*
Content-Type: application/x-www-form-urlencoded; charset=UTF-8
Origin: https://www.proantic.com
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://www.proantic.com/en/monespace.php?mode=pwd
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

change=pwd&lang=en&csrf-token=[CSRF_TOKEN]&mem_old_pwd=[CURRENT_PASSWORD]&mem_pwd=[NEW_PASSWORD
]&mem_conf_pwd=[NEW_PASSWORD]
```

REPORTED RESPONSE BODY

```
["success","Your password has been changed.","success"]
```

The notes then show a current-password error when the earlier password value was no longer current. That error alone does not establish whether the token was valid or invalid.